

BREAKPOINT V3

Rapport technique officiel — correction du bug de balle sous la raquette, extension à 50 niveaux, identité audio arcade, identifiant de build automatique, durcissement PWA/service worker

Dépôt	scuillerjack-oss/breakpoint-v0 (GitHub)
Branche	main
Commit de base (V2 déployée)	49a67e9d4763ec15daadd25a2c99b7303c55a4bb
Commit V3 (ce rapport)	b62b2f3c048a53ddded9a28dbdc8233694ee1863
Identifiant de build affiché (Réglages)	v0.1.0+b62b2f3
Date	24 septembre 2026

Avertissement liminaire — validation technique ≠ validation produit. Ce rapport documente ce qui a été vérifié par des moyens automatisés (tests unitaires, simulations, vérifications de navigateur réel via Playwright, confirmation du pipeline CI/CD au niveau des jobs, vérification du commit réellement déployé). Il ne remplace en rien la bêta humaine sur téléphone physique, seule habilitée à juger de la jouabilité réelle, du ressenti, de la difficulté perçue, du confort de l'audio et du rythme de progression. BREAKPOINT V3 n'est **pas** déclarée validée comme produit à l'issue de ce travail.

1. Rappel de mission et état de départ audité

Le cahier des charges V3 confiait cinq priorités, en repartant explicitement de l'état réel de V2 déjà développé, sauvegardé et déployé — jamais d'une reconstruction. Avant toute modification, l'état réel du projet a été vérifié :

- Chemin réel : /home/user/breakpoint-v0, dépôt Git valide.
- Branche : main, HEAD au commit 49a67e9 ("Ajoute le rapport technique officiel V2"), arbre de travail propre avant intervention.
- Remote : origin = <https://github.com/scuillerjack-oss/breakpoint-v0.git>, confirmé synchronisé (aucun commit local en avance/retard avant le début des travaux).
- Déploiement V2 confirmé au niveau des jobs CI réels (run #8, jobs build et deploy tous deux "success") — la V2 réellement bêta-testée sur téléphone physique correspond bien au code audité ici.

Retour humain sur la vraie V2 (rapporté par l'utilisateur, après résolution de l'incident de cache documenté séparément) : contrôle tactile nettement plus agréable, position de raquette validée, jouabilité générale agréable, envie d'enchaîner plusieurs niveaux. Ces acquis sont protégés dans tout ce qui suit : aucune des cinq priorités V3 ne les retouche sans nécessité stricte.

2. Priorité 1 — bug de balle sous la raquette : diagnostic et correction

2.1. Symptôme remonté

La bêta physique V2 faisait état d'un comportement intermittent : la balle pouvait passer sous la raquette, rester bloquée, ou parfois repasser au-dessus et redevenir jouable de façon peu claire. Consigne explicite du cahier des charges : diagnostiquer la cause racine avant toute correction, ne jamais se contenter d'un contournement (téléportation, respawn forcé).

2.2. Cause racine identifiée

L'algorithme de collision par balayage (`sweptAABB`, `src/engine/collision.js`) est correct par construction : il calcule un temps d'entrée exact et une normale géométrique qui pointe toujours à l'opposé de la composante de vitesse entrante sur l'axe gagnant — vérifié directement, y compris pour un impact sous la raquette. Le bug ne s'y trouvait pas.

Dans `src/engine/simulation.js`, la correction positionnelle (repousser la balle hors de l'objet touché) utilise bien cette vraie normale, pour tous les collideurs sans exception. Mais la réponse en **vitesse** pour le collideur "paddle" était traitée à part : une formule de visée par décalage horizontal (+ effet "English" de la vitesse de raquette, introduit en V1) renvoyait **systématiquement** la balle vers le haut, quelle que soit la normale réelle calculée par `sweptAABB`.

Pour un impact réel sur la face **inférieure** de la raquette (balle déjà passée dessous, typiquement après un rebond de mur), ces deux logiques entraient en contradiction directe : la position était repoussée sous la raquette (normale réelle, vers le bas) tandis que la vitesse était forcée vers le haut (formule de visée, ignorant la normale). La balle repartait donc immédiatement re-percuter la même face au sous-pas suivant — un piège de collision perpétuel. Reproduit de façon déterministe avant correction : la balle restait épinglée à `y == raquette.maxY + rayon + 0.01` avec `vy` inchangée sur des dizaines de ticks consécutifs.

2.3. Correction appliquée (minimale, ciblée)

La formule de visée par décalage n'est désormais réservée qu'aux impacts où `best.normal.y < 0` (face supérieure réelle, seul cas où elle a un sens physique). Tout le reste — face inférieure, côtés — utilise désormais `reflect()` avec la normale réelle, exactement comme n'importe quel autre collideur (mur, brique). Aucune autre logique de jeu n'a été modifiée.

2.4. Preuve de non-régression et de non-vacuité

- 9 nouveaux tests dans `tests/ball-under-paddle.test.js` : absence de position figée, rebond vers le bas sur impact réel sous la raquette, sortie garantie vers une perte normale (jamais bloqué indéfiniment), non-retour à la jouabilité avant un vrai événement de perte, corner-clip à haute vitesse, proximité du bord gauche de l'arène, trajectoire rasante, vitesse très élevée proche du tunneling, et un test de non-régression explicite confirmant qu'un impact réel sur la face supérieure rebondit toujours vers le haut comme en V2.
- Non-vacuité vérifiée empiriquement : le correctif a été temporairement annulé et exactement 5 des 9 nouveaux tests échouaient réellement sur l'ancien code — la suite détecte bien le bug qu'elle prétend couvrir, ce n'est pas une suite qui passerait de toute façon.

3. Priorité 2 — extension à 50 niveaux réels et distincts

3.1. Contrainte de densification documentée

Le cahier des charges impose de privilégier l'ajout de rangées de briques, et de ne réduire la taille/l'espacement des briques qu'en dernier recours documenté si les rangées finissaient par empiéter sur la zone raquette. Le plafond géométrique réel a été calculé à partir des constantes d'arène existantes :

$$\text{MAX_COMFORTABLE_ROWS} = \text{floor}((\text{PADDLE_Y} - \text{BRICK_TOP_Y} - 130) / \text{BRICK_ROW_H}) = 16$$
rangées, avec une marge de confort de 130px avant la zone raquette. Le niveau le plus dense des 50 (niveau 50) utilise 14 rangées — 2 rangées sous ce plafond. **Conclusion documentée** : le row-count seul suffit pour l'intégralité des 50 niveaux ; aucune réduction de taille de brique n'a été nécessaire.

3.2. Motifs générés, composables et variés

Huit générateurs de motifs déterministes (aucun `Math.random`) composent les 40 niveaux générés (11 à 50), au-dessus des 10 niveaux hérités de V0/V1/V2 conservés à l'identique : cadre (creux), damier, colonnes, losange, appareillage (quinconce), dispersion pseudo-aléatoire déterministe, bandes alternées, mur à colonnes espacées. Le style sélectionné à chaque niveau dépend du **contraste de points de vie réellement calculé** entre la valeur principale et la valeur alternative (et non d'un simple index de palier) — cette bascule a été nécessaire après que la première version, gâtée sur l'index de palier seul, produisait par endroits deux niveaux consécutifs structurellement identiques une fois les points de vie réduits par le plafond anti-marathon (voir 3.3).

3.3. Plafond anti-marathon

Une première calibration proportionnelle du temps cible (dérivée du ratio implicite du niveau 10 déjà validé) produisait, une fois combinée à la croissance du nombre de rangées ET du palier de points de vie, des niveaux allant jusqu'à ~800 secondes (~13 minutes) sur les paliers les plus difficiles — exactement le "remplissage artificiel"/"niveaux beaucoup trop longs" explicitement proscrit par le cahier des charges. Corrigé par un système de plafond total d'impacts par palier (`capTotalHits`, paliers [70, 95, 115, 140] impacts), qui réduit itérativement les points de vie des briques les plus résistantes jusqu'à respecter le budget avant de calculer le temps cible final. Résultat mesuré : temps cible maximal ramené à ~269 secondes (~4,5 minutes) sur les niveaux les plus difficiles.

3.4. Audit automatique des 50 niveaux

- `validateAllLevels()` : 0 alerte sur les 50 niveaux (longueur de rangée cohérente, caractères valides, présence de briques, ratio impacts/temps cible raisonnable).
- Test de non-duplication : chaque motif de briques des 50 niveaux est structurellement distinct des 49 autres (comparaison exacte rangée par rangée). Ce test a effectivement détecté et fait corriger 4 collisions réelles pendant le développement (niveaux 17/18, 33/34, 43/44, 49/50), chacune remontant à une cause différente et documentée dans le code.
- `tests/levels-reachability.test.js` (préexistant depuis V1, automatiquement étendu aux 50 niveaux sans code supplémentaire) : chaque niveau est rejoué sous 3 profils de bruit "raquette imparfaite réaliste" distincts et doit se terminer dans une borne généreuse — les 50/50 niveaux passent.
- Capture d'écran réelle (Playwright, 390×844) du niveau 50 : lisibilité visuelle confirmée, zone raquette/pouce toujours préservée.

4. Priorité 3 — identité audio arcade

4.1. Choix architectural : synthèse procédurale, aucun asset externe

L'intégralité de la musique et des effets sonores est générée par synthèse WebAudio pure (oscillateurs, filtres, bruit blanc filtré) — **aucun fichier audio externe n'est chargé ou embarqué**. Ce choix répond directement à l'exigence du cahier des charges (section 6) concernant la provenance et les droits pour une exploitation commerciale Android/Google Play : puisqu'il n'existe aucun asset tiers, il n'existe structurellement aucune question de licence, d'attribution ou de droit à documenter au-delà du code source lui-même, qui est la propriété

entière du projet.

4.2. Musique de fond

- Boucle de 8 mesures en La mineur, progression Am–F–C–G, ~104 BPM (~19,2 s par cycle).
- Deux variantes de motif mélodique différentes (mesures 1-4 / 5-8) pour éviter l'effet "quelques notes pauvres répétées" explicitement proscrit — la répétition de la boucle entière n'est donc jamais note pour note.
- Basse en marche (root-root-quinte-root par mesure), nappe d'accord tenue en onde triangle très douce, mélodie en arpèges consonants sur une octave modérée (ondes sinus/triangle, jamais d'onde carrée pure sur les éléments mélodiques, pour éviter le "aigu fatigant" sur sessions prolongées), tick percussif discret filtré en passe-haut.
- Ordonnancement par *lookahead scheduler* (motif standard WebAudio) : les notes sont planifiées à l'avance sur l'horloge audio elle-même (`AudioContext.currentTime`), jamais sur `setTimeout/Date.now()`, pour une boucle sans dérive ni coupure audible.
- Volume musique nettement subordonné aux effets sonores de gameplay (gain de base 0,16 contre 0,5 pour le SFX de raquette) — ne masque jamais un retour de jeu.

4.3. Contraintes autoplay mobile/PWA respectées

- Démarrage strictement sur un geste utilisateur réel (clic "Jouer"/"Continuer", clic "Reprendre", ou le clic de la case à cocher Musique elle-même) — jamais au chargement de la page.
- Garde interne empêchant toute instance superposée (`if (!musicEnabled || musicTimer) return;`).
- Arrêt propre à la mise en pause et à la perte de focus (onglet en arrière-plan) ; reprise propre sur un nouveau geste réel, jamais automatique hors geste.
- Réglage persistant (case à cocher "Musique" dans les Réglages, clé `settings.music` déjà présente dans le schéma de sauvegarde depuis V0, aucune migration nécessaire).

4.4. Effets sonores (SFX)

Les effets sonores courts et lisibles préexistants depuis V0/V1 (raquette, brique simple, brique renforcée, destruction, **bonus/power-up** avec glissando montant reconnaissable, perte de balle, victoire) couvrent déjà l'exigence "SFX courts et lisibles, en particulier pour les bonus" — ils n'ont pas été dupliqués ni ajoutés inutilement, conformément à la consigne de ne pas ajouter de contenu superflu sans bénéfice réel démontré.

5. Priorité 4 — identifiant de build automatique

Directement motivé par l'incident de cache V2 (un déploiement à jour n'était pas distinguable d'un ancien depuis l'appareil de test). `vite.config.js` calcule désormais, à chaque build, un identifiant `vX.Y.Z+hash` combinant la version de `package.json` et le hash court du commit **réellement construit** — `GITHUB_SHA` (fourni par le runner CI, prioritaire car un clone CI peut être "shallow") en environnement CI, ou `git rev-parse --short HEAD` en local. Aucune valeur codée en dur, aucune maintenance manuelle possible.

Affiché discrètement en bas de l'écran Réglages (texte à faible opacité, sélectionnable), facilement vérifiable sans gêner le gameplay. Un test automatisé (Playwright) vérifie que la valeur affichée correspond au format attendu ET au commit réellement construit — sa non-vacuité a été prouvée en injectant délibérément un hash de commit erroné et en confirmant que le test échoue bien dans ce cas, exactement le scénario de l'incident V2.

6. Priorité 5 — durcissement PWA / service worker / cache

La stratégie réseau-d'abord introduite en V2 reste inchangée et n'a pas été remise en cause : elle rend déjà structurellement impossible qu'une réponse périmée soit servie tant que le réseau répond. Deux angles morts résiduels ont été identifiés et corrigés :

- **Onglet PWA laissé longtemps ouvert** : le navigateur ne revérifie une mise à jour du service worker qu'au hasard d'une navigation, avec un débounce interne documenté (~24h). `main.js` appelle désormais `registration.update()` à chaque retour réel au premier plan (événement `visibilitychange`) — cela ne supprime pas la limite du navigateur, mais maximise les occasions réelles de la déclencher.
- **Onglet déjà ouvert qui ne se rafraîchit jamais tout seul** : même quand une mise à jour est détectée et prend le contrôle (`skipWaiting + clients.claim()`, déjà en place depuis V2), un onglet déjà ouvert continuait à exécuter en mémoire le JS de l'**ancien** build indéfiniment. Un listener `controllerchange` déclenche désormais un rechargement automatique, unique (garde anti-boucle), dès qu'un nouveau service worker prend réellement le contrôle.
- Purge de confort des entrées de cache d'assets hachés au-delà de 30 entrées, pour éviter une accumulation de stockage illimitée d'un build à l'autre (mesure d'hygiène, sans impact sur la fraîcheur déjà garantie par la stratégie réseau-d'abord).

Vérification non-vacuité : un test end-to-end (Playwright) modifie réellement le contenu du `sw.js` servi (simulant un vrai nouveau déploiement), simule un retour au premier plan, et vérifie que la page se recharge automatiquement — le test a été confirmé défaillant lorsque le listener `controllerchange` était retiré, avant d'être restauré.

7. Résultats de tests — régression complète

Tests unitaires (node:test)	98/98 réussis (0 échec) — inclut les 9 nouveaux tests de collision balle/raquette et les tests étendus de niveaux (50 niveaux, non-duplication, plafond d'impacts)
Vérifications mobiles réelles (Playwright)	21/21 réussies — 17 préexistantes (V0-V2, aucune régression) + 4 nouvelles V3 (cycle de vie musique/AudioContext unique, réglage musique persistant, identifiant de build affiché et exact, rechargement automatique après mise à jour du service worker)
Performance sous stress	~60,0 fps moyen, 59,5 fps minimum, 0 frame sous 30fps sur 235 frames testées, 0 erreur console — inchangé par rapport à V2
Audit des 50 niveaux	validateAllLevels() : 0 erreur ; 50/50 motifs distincts ; 50/50 réellement terminables sous 3 profils de bruit réalistes

Chaque nouvelle suite de tests critique a été soumise à une preuve de non-vacuité explicite (annulation temporaire du correctif correspondant, confirmation que les tests concernés échouent réellement, puis restauration du correctif) : le bug de balle sous la raquette (5/9 tests échouant sans le correctif), le cycle de vie musique/pause (2/2 tests échouant sans l'arrêt à la mise en pause), l'identifiant de build (échec confirmé avec un hash injecté délibérément erroné), et le rechargement automatique du service worker (échec confirmé sans le listener controllerchange).

8. État du dépôt Git et vérification du déploiement

Commit local créé	b62b2f3c048a53ddded9a28dbdc8233694ee1863
Push	git push -u origin main → 49a67e9..b62b2f3 main -> main
Vérification distante indépendante	git ls-remote origin refs/heads/main → b62b2f3c048a53ddded9a28dbdc8233694ee1863 (identique au commit local — confirmé, pas supposé)
Run CI déclenché	"Deploy to GitHub Pages" run #9 (id 36064731157), event push, head_sha = b62b2f3c... (exact)
Job "build"	success — étapes détaillées : npm ci ✓, npm test ✓ (98/98 dans le runner réel), npm run build ✓, playwright install ✓, npm run test:mobile ✓ (21/21 dans le runner réel), upload-pages-artifact ✓
Job "deploy"	success — actions/deploy-pages@v4 ✓
Vérification du contenu réellement servi	Limite d'environnement documentée depuis V0 : ce bac à sable ne peut pas atteindre *.github.io directement (proxy réseau, EGRESS_BLOCKED confirmé sur cette tentative). Contournement appliqué : confirmation au niveau des jobs CI réels (pas seulement le statut global du workflow) + reconstruction locale avec GITHUB_SHA=b62b2f3c... donnant l'identifiant exact v0.1.0+b62b2f3, identique à ce que le build CI a produit.

9. Identifiant de build que l'utilisateur doit voir sur son téléphone

Après ouverture de l'app puis **Réglages**, l'identifiant affiché en petit texte discret en bas de l'écran doit être exactement :

v0.1.0+b62b2f3

Si un appareil affiche un identifiant différent (ou l'ancien comportement V1/V2) après avoir rouvert l'application, c'est le signal direct et sans ambiguïté d'un service worker non encore mis à jour sur cet appareil précis — voir section 6 pour les mécanismes de rafraîchissement désormais en place, et rappeler qu'un rechargement manuel complet reste toujours le recours immédiat.

10. Ce qui reste explicitement non validé

Conformément à la méthodologie exigée par le cahier des charges, ce rapport ne déclare **à aucun moment** BREAKPOINT V3 validée comme produit. Restent exclusivement du ressort d'une bêta humaine sur téléphone physique :

- Le ressenti réel de jouabilité sur les 50 niveaux (rythme, difficulté perçue, fatigue éventuelle) — les simulations automatisées prouvent la terminabilité, pas le plaisir de jeu.
- Le confort d'écoute réel de la musique de fond sur une session prolongée (les tests automatisés prouvent le cycle de vie technique — démarrage, arrêt, absence de superposition — pas l'agrément subjectif).
- La confirmation que le bug de balle sous la raquette ne se manifeste plus du tout en usage réel prolongé, au-delà des scénarios de collision reproduits par les tests.
- La confirmation que l'identifiant de build affiché correspond bien à la version reçue sur l'appareil physique de test après un vrai cycle d'installation/mise à jour PWA.

Rapport généré à partir de vérifications automatisées et de l'état réel du dépôt/CI/CD au 24 septembre 2026. Aucune affirmation de validation produit n'est faite dans ce document.